Soft Recovery and Record Capacities in Nested Elimination

Hans Brende GPT-6 Astra (OpenAI)

October 2026

Abstract

We extend the decomposition theorem of *Nested Elimination and Half-Rate Bounds for Storage Codes* to recovery with nonzero conditional uncertainty. Every certified decomposition gives a half-rate inequality with an explicit residual term, retaining the conditioning supplied by earlier pieces. We then account for additional helper records by their actual occurrences and conditioning contexts. A reused helper pays for its terminal mutual information and positive information losses between uses; literal nesting of contexts is unnecessary. Combining these bounds with exact revelation costs and coordinate credits gives one inequality that permits correlated nonlinear records, discarded information, and incomplete recovery. Simpler capacity bounds, including a coefficient $\min\{2, k_e\}$ for a record covered by k_e retained lists, follow as corollaries. All new numbered results are proved in Lean. Finite entropy examples establish sharp coefficients and show why the refined loss must be evaluated after revelation. The memo supplies no new uniform tape transposition lower bound; extracting affordable certificates from actual computations remains open.

Contributions. Hans Brende directed the research and selected the questions. GPT-6 Astra (OpenAI) developed the extensions, Lean proofs, and text under his direction.

Contents

1 Introduction **2**

2 Ranks and decomposition certificates **2**

3 Soft decomposition **4**

4 Information supplied by a reused helper **5**

5 Record bounds and revelations **7**

6 Sharp coefficients and finite controls **10**

7 Use in the transposition program **12**

A Index of Lean declarations **13**

B Verification and reuse **13**

1 Introduction

A storage code prescribes, for each coordinate v, a set $N(v)$ of coordinates from which its value is determined. The seventh paper of the series proves a half-rate bound whenever N has a certified piece decomposition [3]. Its proof applies to every normalized, monotone, submodular rank function satisfying those exact recovery equalities. The tenth paper constructs these certificates for the directed class called Class S [4].

In a computation, a useful relation may account for only part of a local recovery. Additional information can reside in coded records, be shared by several recoveries, or be lost when the available context changes. The purpose of this memo is to retain a certified relation while charging those remaining obligations explicitly.

There are three steps. Theorem 3.1 keeps the recovery uncertainty in the decomposition argument. Lemma 4.1 bounds the information supplied by one helper across arbitrary contexts. Theorem 5.3 combines these statements with a cover of the actual helper occurrences. The resulting bound keeps the information revealed about the selected input, unused coordinate capacity, terminal helper information, and the remaining recovery uncertainty. Its simpler capacity corollaries expose which quantities a concrete application must control.

Formal proofs. Every new numbered theorem, lemma, corollary, and proposition below is proved in the accompanying Lean library. Those declarations are the proofs; the English statements and proof outlines are guides to them. The finite Shannon instances reuse the entropy library of the earlier papers [5, 1]. The general inequalities hold for all polymatroids, not only entropic ones. No numerical experiment or unformalized intermediate claim is used to prove a result. The application discussion identifies obligations still to be met; it is not a formalized transposition lower bound.

Relation to earlier work. Entropy and functional dependence are established tools for network coding bounds; see, for example, Thakor, Grant, and Chan [7]. Elishco and Barg study recoverable systems whose local conditional entropy is bounded by a positive tolerance [6, Section IV]. Thus neither the use of conditional entropy to describe imperfect recovery nor functional dependence as a form of information retention is new in itself. The contribution here is a residual extension of the seventh paper's specific decomposition engine, together with exact occurrence accounting for helper reuse and revelation. We claim neither a new class of exact half-rate relations nor a separation from all earlier information bounds.

The relevance to the transposition program of the first paper [2] is conditional: these tools can certify an impossibility only when the available coordinate capacity exceeds all the charged information. Section 7 states that remaining task and the constraints it places on constructions.

2 Ranks and decomposition certificates

All coordinate and record sets are finite. A *polymatroid* is a function $h : 2^A \to \mathbb{R}$ with $h(\varnothing) = 0$, monotonicity under inclusion, and

$$h(S \cup T) + h(S \cap T) \le h(S) + h(T).$$

Write

$$h(S \mid K) = h(S \cup K) - h(K),$$
$$I_h(S; T \mid Z) = h(S \mid Z) - h(S \mid T \cup Z),$$
$$h_Z(S) = h(S \mid Z).$$

A singleton may be written without braces. Contraction h_Z and pullback along any map $f : V \to A$ are again polymatroids. The map need not be injective. These are the existing contraction theorem and the memo's `image_isPolymatroid` and `conditionalImage_isPolymatroid`.

For jointly distributed finite random variables W_a, Shannon entropy $h(S) = H(W_S)$ supplies an instance. We use natural logarithms, as does the Lean entropy library: a fair bit has entropy $\log 2$. All terms must use the same units. Symbols such as $H(L \mid K, Z)$ abbreviate the entropy of the corresponding tuples. No independence is implicit.

The imported structural certificate. Let V be a finite vertex set and $N : V \to 2^V$ a prescribed-parent relation. We recall the certificate of [3] to make clear which hypothesis is retained. A piece $H \subseteq V \setminus U$ has a certificate $\mathsf{PieceCert}(N, U, H)$ consisting of:

(i) finitely many pairwise disjoint sets $P_1, \ldots, P_m$ whose union is H;

(ii) a list ℓ whose set of entries is H, with repetitions allowed, and maps $a, b : V \to \{1, \ldots, m\}$ such that $v \in P_{a(v)}$ and $a(v) \neq b(v)$ for every entry;

(iii) a fixed optional outside parent $\mathrm{par}(v)$ for each vertex, satisfying the following scan conditions.

The scan starts with $S = \varnothing$ and $M_w = \varnothing$ for every $w \in V$. At an entry v, put $K = S \cup P_{b(v)}$. If there is no outside parent, require $N(v) \subseteq U \cup K$. If the outside parent is w, require

$$w \in V \setminus (U \cup H), \qquad N(v) \subseteq U \cup K \cup \{w\}, \qquad M_w \subseteq K,$$

and update M_w to $K \cup \{v\}$. In both cases insert v into S. The supplying part $P_{b(v)}$ is part of this structural certificate; it is distinct from the actual helper records introduced later.

An ordered list of pieces $\mathcal{H} = (H_1, \ldots, H_t)$ satisfies $\mathsf{Decomp}(N, U, \mathcal{H})$ when the pieces partition $V \setminus U$ and each H_j has a piece certificate in context

$$U_j = U \cup \bigcup_{i<j} H_i.$$

This is precisely the imported predicate `HalfRate.Decomp`, with piece type `HalfRate.PieceCert`. These definitions contain no rank function or recovery equality. They are not inferred merely from the chronological order of a tape computation.

Class S. An arc $u \to v$ means $u \in N(v)$. Class S consists of loopless relations with no transitive triangle on three distinct vertices, in-degree at most three, and the property that every vertex of in-degree three reciprocates each outgoing arc. The relative condition on $R \subseteq V$ imposes these requirements on the induced relation inside R, counting only parents in R. The tenth paper proves that this relative condition supplies $\mathsf{Decomp}(N, V \setminus R, \mathcal{H})$ for some $\mathcal{H}$ [4, Theorem S*]; its proof is Lean-verified there.

3 Soft decomposition

Theorem 3.1 (S: residual decomposition)**.** Lean: `decomp_ineq_residual`, `decomp_ineq_soft` [`Decomposition.lean`]

Let $h : 2^V \to \mathbb{R}$ be a polymatroid and let $\mathsf{Decomp}(N, U, (H_1, \dots, H_t))$*. With U_j as above,*

$$2\big(h(V) - h(U)\big) \le \sum_{v \notin U} h(v \mid U) + \sum_{j=1}^{t} \sum_{v \in H_j} h(v \mid N(v) \cup U_j) \tag{3.1}$$

$$\le \sum_{v \notin U} h(v \mid U) + \sum_{v \notin U} h(v \mid N(v) \cup U). \tag{3.2}$$

No recovery equality is assumed.

Outline of the formal proof. In a piece, work with $g = h_U$. A scan step with supplying context $K = S \cup P_{b(v)}$ and outside parent w has the exact identity

$$g(v \mid K) = I_g(w; K \cup \{v\}) - I_g(w; K) + g(v \mid K \cup \{w\}).$$

The last term is at most $h(v \mid N(v) \cup U)$ by the parent containment condition. A step without an outside parent has the same residual bound directly. If v has already been scanned, its increment is zero; repetitions are therefore not charged again. The nested outside-parent potentials telescope, giving the piece bound

$$2h(H \mid U) \le \sum_{v \in H} h(v \mid U) + \sum_{w \notin U \cup H} I_h(w; H \mid U) + \sum_{v \in H} h(v \mid N(v) \cup U).$$

The original elimination inequality supplies its first two terms; `totalDefect_le_residual` and `piece_ineq_residual` keep the residuals. Induct over pieces. The identity $h(w \mid U \cup H) = h(w \mid U) - I_h(w; H \mid U)$ cancels the outside potentials with the next piece's singleton terms. This proves (3.1). Conditional monotonicity and the disjoint partition of $V \setminus U$ give (3.2). □

The retained contexts U_j can reduce the residual bill. The simpler form is useful when only a uniform estimate before the piece scan is available. Both allow uncertified dependencies to be paid as remaining uncertainty while keeping the same certified relation N.

Corollary 3.2 (Common observations)**.** Lean: `conditional_soft`, `entropy_conditional_soft` [`Observations.lean`]

Let $X_v = W_{f(v)}$, let Z be any common tuple of ambient variables, and suppose $\mathsf{Decomp}(N, \varnothing, \mathcal{H})$*. Then*

$$2H(X_V \mid Z) \le \sum_{v} H(X_v \mid Z) + \sum_{v} H(X_v \mid X_{N(v)}, Z). \tag{3.3}$$

The law is arbitrary on a finite sample space and finite alphabet. The same inequality holds for conditional ranks of any ambient polymatroid under any coordinate map f.

Outline of the formal proof. Apply Theorem 3.1 to the polymatroid $S \mapsto h(f(S) \cup Z) - h(Z)$ with $U = \varnothing$. The finite entropy theorem substitutes the existing `MotionDepthCodes.codeEntropy` instance. □

Thus common information may include nonlinear observations and correlated auxiliary variables; independent input coordinates are not required.

Corollary 3.3 (Exact recovery)**.** Lean: `exact_decomp`, `exact_rateBound`, `exact_card_sq` [`Decomposition.lean`]

Under the hypotheses of Theorem 3.1, if $h(v \mid N(v)) = 0$ for all v, the residuals vanish and

$$2\big(h(V) - h(U)\big) \le \sum_{v \notin U} h(v \mid U).$$

In particular a decomposition from $\varnothing$ gives the original half-rate bound. For every storage code $C \subseteq \alpha^V$ over a finite nonempty alphabet, it gives $|C|^2 \le |\alpha|^{|V|}$. Here a storage code means that agreement on $N(v)$ determines agreement at v for every pair of words in C.

Outline of the formal proof. Nonnegativity and conditional monotonicity force every residual to zero. The original rate-to-cardinality theorem then supplies the cardinality claim, including the empty-code case. □

This derives the earlier exact theorem directly from the extension.

Corollary 3.4 (Relative Class S)**.** Lean: `classSOn_soft`, `classS_soft` [`ClassS.lean`]

If N satisfies relative Class S on $R \subseteq V$ and h is any polymatroid, then

$$2\big(h(V) - h(V \setminus R)\big) \le \sum_{v \in R} h(v \mid V \setminus R) + \sum_{v \in R} h(v \mid N(v) \cup (V \setminus R)).$$

Taking $R = V$ gives the global soft inequality on Class S.

Outline of the formal proof. Use the tenth paper's relative decomposition certificate and apply Theorem 3.1 with $U = V \setminus R$. □

Existing Class S certificates therefore remain usable with explicit nonzero recovery uncertainty; no additional graph classification is claimed.

4 Information supplied by a reused helper

Fix a common context C and one helper variable Y_e. A use is a pair (K_i, x_i): K_i is the set of available variable labels and W_{x_i} is the target. The target need not be fully recoverable. Its *innovation* from the helper is

$$\iota_i = I(Y_e; W_{x_i} \mid W_{K_i}, W_C) = I(Y_e; W_{K_i \cup \{x_i\}} \mid W_C) - I(Y_e; W_{K_i} \mid W_C). \tag{4.1}$$

For readability, subsequent expressions suppress W on contexts.

For a list of r uses and an initial set L_0, put $L_1 = L_0$ and $L_i = K_{i-1} \cup \{x_{i-1}\}$ for $2 \le i \le r$. Its end context is $K_{\text{end}} = K_r \cup \{x_r\}$ when $r > 0$, and L_0 otherwise. This end context includes the target as an analysis variable; it does not assert that an incomplete decoder has obtained it. Define

$$D_e(C) = \sum_{i=1}^{r} \big[I(Y_e; L_i \mid C) - I(Y_e; K_i \mid C)\big]_+ , \tag{4.2}$$

$$\mathcal{L}(C) = \sum_{i=1}^{r} H(L_i \mid K_i, C), \tag{4.3}$$

where $[t]_+ = \max\{0, t\}$. The first sum measures decreases in information about this particular helper; the second charges all information lost from the preceding context and target.

Lemma 4.1 (Helper potential)**.** Lean: `innovations_le_helperLoss`, `helperLoss_le_chainLoss`, `innovations_le_helper_capacity`, `innovations_le_with_loss` [`Innovation.lean`]

For every list of uses,

$$\sum_{i=1}^{r} \iota_i \le I(Y_e; K_{\mathrm{end}} \mid C) - I(Y_e; L_0 \mid C) + D_e(C). \tag{4.4}$$

Moreover $0 \le D_e(C) \le \mathcal{L}(C)$*, and hence the right side may be replaced by either*

$$I(Y_e; K_{\mathrm{end}} \mid C) - I(Y_e; L_0 \mid C) + \mathcal{L}(C) \quad \text{or} \quad H(Y_e \mid L_0, C) + D_e(C).$$

The first inequality is purely algebraic: with the definitions in Section 2, it holds for every real-valued set function, without the polymatroid axioms. The comparisons with conditional entropy and helper capacity require those axioms.

Outline of the formal proof. Set $p(S) = I(Y_e; S \mid C)$. Summing $\iota_i = p(K_i \cup \{x_i\}) - p(K_i)$ and inserting the intervening $p(L_i)$ gives the exact identity

$$\sum_i \iota_i = p(K_{\mathrm{end}}) - p(L_0) + \sum_i (p(L_i) - p(K_i)).$$

Replace each last summand by its positive part. For a polymatroid, monotonicity and submodularity give $p(L_i) - p(K_i) \le H(L_i \mid K_i, C)$, proving the comparison of losses. Finally $p(K_{\mathrm{end}}) \le H(Y_e \mid C)$ gives the capacity form. The Lean proof implements the telescoping by list induction. □

This permits information discarded between helper uses to be charged without requiring the whole context to be retained.

Lemma 4.2 (Retention through an encoding)**.** Lean: `retained_innovations_le`, `NestedUses.retained`, `RetainedUses.contr`, `chainLoss_contr_le` [`Innovation.lean`]

Suppose $H(L_i \mid K_i, C) = 0$ *at every transition. Then*

$$\sum_i \iota_i \le H(Y_e \mid L_0, C).$$

Literal containment $L_i \subseteq K_i$ *is sufficient. More generally, it suffices that the next context determines the preceding context and target, possibly under different variable names. Zero conditional entropy, and the resulting retention condition, survive any additional common conditioning. The whole-context loss* $\mathcal{L}$ *can only decrease under that conditioning.*

Outline of the formal proof. The predicate `RetainedUses` requires exactly the displayed zero conditional entropies. They set the loss sum to zero in Lemma 4.1. Inclusion implies zero conditional rank. Conditional monotonicity proves preservation under common revelation and the comparison of loss sums. An actual deterministic decoder certifies zero conditional entropy by `entropy_conditionalRank_zero_of_recovery`; its proof uses equality of the fibers of the finite observations. □

The assertion concerns information retained, not a decoder's running time or physical availability. It does not license adding variables to an actual use context. The smaller loss D_e does *not* share the conditioning monotonicity of $\mathcal{L}$; see Proposition 6.3.

Lemma 4.3 (Ordered helpers)**.** Lean: `increment_eq_innovation_add`, `increment_eq_helperInnovations_add` [`Innovation.lean`]

For a target X and an ordered list of helpers $(Y_{e_1}, \dots, Y_{e_m})$,

$$H(X \mid K, C) = \sum_{j=1}^{m} I(Y_{e_j}; X \mid K, Y_{e_1}, \dots, Y_{e_{j-1}}, C) + H(X \mid K, Y_{e_1}, \dots, Y_{e_m}, C). \tag{4.5}$$

The identity also holds for conditional ranks of any set function. Repeated helper identities are allowed.

Outline of the formal proof. Subtract the conditional target entropy before and after one helper, then telescope over the list. A repeated helper contributes zero once its variable is already present in the conditioning set. □

This fixes the contexts that an occurrence cover must preserve. Charging the same helper in a larger, more convenient context would not be the expansion of the given local recovery.

5 Record bounds and revelations

Let A label all ambient variables W_a, let $X_v = W_{f(v)}$ for $v \in V$, and write $Y_e = W_e$ when label e serves as a helper. Fix a common context $Z \subseteq A$, a relation N, and a certificate $\mathsf{Decomp}(N, \varnothing, \mathcal{H})$. For each v choose an ordered list of helper labels $F_v = (e_{v,1}, \dots, e_{v,m_v})$. Its jth occurrence is the triple

$$(e_{v,j},\ K_{v,j},\ f(v)), \qquad K_{v,j} = f(N(v)) \cup \{e_{v,1}, \dots, e_{v,j-1}\}. \tag{5.1}$$

All conditioning on Z is applied to the rank function, so it is not duplicated inside $K_{v,j}$.

Definition 5.1 (Occurrence cover)**.** Lean: `RecordUseCover`, `recordUses`, `allRecordUses`, `coveredUses` [`ChainCapacity.lean`]

A record-use cover assigns to each label e a finite list $\mathcal{C}_e$ of use lists for that helper. The multiset of triples obtained by tagging every use in every list with its helper e must equal the multiset of all triples in (5.1). Set $k_e = |\mathcal{C}_e|$. Each use list starts with $L_0 = \varnothing$.

The Lean field proving this equality is a permutation of lists. It preserves identities, contexts, targets, and multiplicities, including repeated occurrences. The definition imposes neither retention nor an entropy bound. A cover always permits the coarse option of singleton use lists; finding a cover with an affordable bill is a separate task.

Put $B = \sum_v b_v$ for real numbers b_v, and assume

$$H(X_V \mid Z) \geq B - \delta. \tag{5.2}$$

For any set $Q \subseteq A$ revealed in common, write $C_Q = (Z, Y_Q)$ and define

$$S_Q = \sum_v (b_v - H(X_v \mid C_Q)), \qquad J_Q = I(X_V; Y_Q \mid Z), \tag{5.3}$$

$$\Phi_Q = \sum_e \sum_{\gamma \in \mathcal{C}_e} I(Y_e; K_{\mathrm{end}}(\gamma) \mid C_Q), \qquad D_Q = \sum_e \sum_{\gamma \in \mathcal{C}_e} D_e(C_Q; \gamma), \tag{5.4}$$

$$\mathcal{L}_Q = \sum_e \sum_{\gamma \in \mathcal{C}_e} \mathcal{L}(C_Q; \gamma), \qquad E_Q = \sum_v H(X_v \mid X_{N(v)}, Y_{F_v}, C_Q). \tag{5.5}$$

Sums over $\mathcal{C}_e$ count lists with multiplicity. The definitions of D_e and $\mathcal{L}$ are (4.2) and (4.3). The end context of an empty list is empty and contributes zero. The quantity S_Q is unused coordinate capacity; J_Q is the information disclosed about the selected input.

Lemma 5.2 (Revelation and coordinate credit)**.** Lean: `capacity_bound_with_credit`, `revealCredit_nonneg`, `capacity_bound_of_residual` [`Capacity.lean`]

Under the decomposition and lower-entropy assumptions above, if

$$\sum_v H(X_v \mid X_{N(v)}, C_Q) \le R,$$

then

$$B + S_Q \le 2\delta + 2J_Q + R. \tag{5.6}$$

No individual upper bound on $H(X_v \mid Z)$ is required here. If $H(X_v \mid Z) \le b_v$ for all v, then $S_Q \ge 0$ and consequently

$$B \le 2\delta + 2H(Y_Q \mid Z) + R.$$

Outline of the formal proof. The exact entropy drop is $H(X_V \mid C_Q) = H(X_V \mid Z) - J_Q$. Apply Corollary 3.2 after the reveal and substitute $\sum_v H(X_v \mid C_Q) = B - S_Q$. This gives $2(B - \delta - J_Q) \le B - S_Q + R$. Conditional monotonicity gives $S_Q \ge 0$ under the additional singleton bounds, and $J_Q \le H(Y_Q \mid Z)$ gives the last inequality. □

This preserves the distinction between a reveal's entropy and the information it gives about the selected coordinates.

Theorem 5.3 (I: information bound)**.** Lean: `RecordUseCover.information_bound`, `RecordUseCover.residual_information_bound` [`ChainCapacity.lean`]

For every occurrence cover, every common reveal Q, and the decomposition and lower-entropy assumptions above,

$$\boxed{B + S_Q \le 2\delta + 2J_Q + \Phi_Q + D_Q + E_Q.} \tag{5.7}$$

Exact recovery, retention, and individual capacity bounds on b_v are not assumptions. The theorem holds for arbitrary ambient polymatroids as well as finite Shannon entropy.

Outline of the formal proof. Use Lemma 4.3 for every vertex, in common context C_Q. Exact occurrence coverage preserves the total innovation sum when regrouping by helper and use list. Apply Lemma 4.1 to each list. Since $L_0 = \varnothing$, its initial mutual information is zero. This bounds the summed recovery uncertainty by $\Phi_Q + D_Q + E_Q$. Substitution in Lemma 5.2 proves the result. □

The accounting permits coding and partial retention, but still requires the prescribed relation and every context in (5.1) to be justified. It is a necessary information inequality, not a construction of such a certificate from a tape computation.

Corollary 5.4 (Entropy and capacity forms)**.** Lean: `RecordUseCover.entropy_bound`, `chain_record_bound_with_loss`, `optimized_chain_record_bound_with_loss` [`ChainCapacity.lean`]

In addition suppose $H(X_v \mid Z) \le b_v$. For every Q,

$$B \le 2\delta + 2H(Y_Q \mid Z) + \sum_e k_e H(Y_e \mid C_Q) + \mathcal{L}_Q + E_Q. \tag{5.8}$$

If also $H(Y_e \mid Z) \le c_e$ for every ambient label e, then

$$B \le 2\delta + 2\sum_{e\in Q} c_e + \sum_{e\notin Q} k_e c_e + \mathcal{L}_\varnothing + E_\varnothing, \tag{5.9}$$

$$B \le 2\delta + \sum_{e} c_e \min\{2, k_e\} + \mathcal{L}_\varnothing + E_\varnothing. \tag{5.10}$$

The empty-subscript quantities are conditioned on Z but on no additional reveal.

Outline of the formal proof. For a fixed Q, discard the nonnegative credit and use

$$J_Q \le H(Y_Q \mid Z), \qquad \Phi_Q \le \sum_e k_e H(Y_e \mid C_Q), \qquad D_Q \le \mathcal{L}_Q.$$

This gives (5.8). Subadditivity bounds the reveal by the sum of its capacities; a revealed helper has zero remaining conditional entropy. The whole-context loss and final recovery uncertainty decrease under added common conditioning, giving (5.9). Choose $Q = \{e : k_e \ge 2\}$. The exact sum identity `threshold_bill` gives (5.10). □

Thresholding minimizes this separable capacity bill. It does not assert optimality for the coupled information terms of (5.7).

Corollary 5.5 (C: retained uses)**.** Lean: `RecordUseCover.retained_entropy_bound`, `optimized_retained_chain_record_bound`, `optimized_chain_record_bound` [`ChainCapacity.lean`]
Assume the hypotheses of Corollary 5.4, exact local recovery $H(X_v \mid X_{N(v)}, Y_{F_v}, Z) = 0$, and semantic retention $H(L_i \mid K_i, Z) = 0$ at every transition of every use list. Then

$$B \le 2\delta + 2H(Y_Q \mid Z) + \sum_e k_e H(Y_e \mid Z, Y_Q) \quad \text{for every } Q,$$

$$B \le 2\delta + \sum_e c_e \min\{2, k_e\}.$$

In particular these bounds hold under literal nesting of every list.

Outline of the formal proof. Retention makes $\mathcal{L}_\varnothing = 0$ and exact recovery makes $E_\varnothing = 0$. Both properties survive common conditioning. Apply Corollary 5.4. Literal nesting implies semantic retention, so the original chain-count theorem is a direct corollary. □

One retained list pays at most one helper capacity, regardless of how many occurrences it contains. A record with several lists can instead be revealed in common, paying at most twice its capacity.

Proposition 5.6 (Helpers without a use cover)**.** Lean: `mixed_record_entropy_bound`, `mixed_record_bound`, `optimized_record_bound` [`Capacity.lean`]
Let each F_v now be a set of helper labels, without a specified cover. Keep the decomposition, lower-entropy assumption, and $H(X_v \mid Z) \le b_v$. For every Q,

$$B \le 2\delta + 2H(Y_Q \mid Z) + \sum_v \big(H(Y_{F_v} \mid X_{N(v)}, C_Q) + H(X_v \mid X_{N(v)}, Y_{F_v}, C_Q)\big). \tag{5.11}$$

If recovery after the helpers is exact and the unconditional bounds $H(X_v) \le b_v$, $H(Y_e) \le c_e$ hold, then, writing $m_e = |\{v : e \in F_v\}|$,

$$B \le 2\delta + 2\sum_{e\in Q} c_e + \sum_v \sum_{e\in F_v\setminus Q} c_e,$$

$$B \le 2\delta + \sum_e c_e \min\{2, m_e\}.$$

Outline of the formal proof. For each vertex, its recovery uncertainty is at most the helpers' joint conditional entropy plus the uncertainty left after those helpers. Use Lemma 5.2 in its entropy form. In the exact case, remove already revealed helpers, bound the others by their individual capacities, and count occurrences by m_e. Choosing $Q = \{e : m_e \geq 2\}$ gives the last inequality. □

This offers a coarser certificate when useful retained lists are not available. In the joint-entropy version, correlated helpers need not pay the sum of their separate capacities.

Lemma 5.7 (Supplying the initial entropy bound)**.** Lean: `conditional_lower_from_deficit` [`Capacity.lean`]

Let I, C, O, G be sets of ambient variable labels with $I \subseteq C \cup O$, and let $M, B, \Delta, g \in \mathbb{R}$. If

$$H(W_I) \geq M - \Delta, \qquad H(W_O) \leq M - B, \qquad H(W_G) \leq g,$$

then

$$H(W_C \mid W_O, W_G) \geq B - (\Delta + g).$$

The same statement holds for any polymatroid.

Outline of the formal proof. Monotonicity gives $h(I) \leq h(C \cup O \cup G)$, and subadditivity gives $h(O \cup G) \leq h(O) + h(G)$. Subtract and substitute the three bounds. No independence or disjointness is used. □

For selected input coordinates, this supplies (5.2) with $Z = (W_O, W_G)$ and $\delta = \Delta + g$, even under a correlated correctness-set distribution.

6 Sharp coefficients and finite controls

The following examples are exact finite entropy computations. They test the accounting rules, not the existence of a native tape transposer. Write $\lambda = \log 2$ for the entropy of a fair bit.

Proposition 6.1 (Necessary coefficients)**.** Lean: `oneBit_decomposable`, `residual_bound_attained`, `residual_coefficient_at_least_one`, `private_helper_recovers`, `private_helper_coefficient_at_least_one`, `shared_helper_coefficient_at_least_two`, `shared_helper_one_fails` [`Sharpness.lean`]

The following universal coefficients cannot be reduced:

(i) the coefficient 1 *on the residual sum in* (3.3)*;*

(ii) the coefficient 1 *when a single private helper pays for a single recovered fair bit;*

(iii) the coefficient 2 *on an unrestricted shared helper that serves the two off-diagonal obligations of the earlier XOR row protocol.*

For (iii), at every integer width $b > 0$ the protocol is correct on the full 2×2 input cube, has no message cost, and has a helper of entropy $b\lambda$. Any bound $2b \leq c\,H(Y)/\lambda$ for these obligations therefore requires $c \geq 2$.

Outline of the formal proof. For (i), take the empty relation on one coordinate and let that coordinate be a fair bit. The relation has a certified decomposition; the two sides of the soft bound are both 2λ. A coefficient c on the residual would require $2\lambda \leq \lambda + c\lambda$. For (ii), take the private helper to be the bit itself, so exact recovery costs λ. For (iii), reuse the full-cube XOR protocol and its entropy calculation from [5]: a shared parity of two independent b-bit off-diagonal words lets

each row recover the other word. Its correctness and helper entropy are checked by the imported `TransposeFollowup.SeparatorSharpness` declarations included in the audit. Substitution gives $2b \le cb$. □

These examples constrain universal capacity coefficients. They do not preclude savings from the more detailed quantities in Theorem 5.3.

Proposition 6.2 (Retention is a strict relaxation)**.** Lean: `copied_contexts_retained`, `copied_contexts_not_nested`, `xor_uses_not_nested`, `xor_uses_not_retained` [`Sharpness.lean`]

There is a finite entropy instance and a use list satisfying semantic retention but not literal nesting. Conversely, the two XOR uses with contexts $\{B\}$ and $\{A\}$, targeting A and B respectively, fail semantic retention whenever $H(B \mid A) > 0$; they also fail literal nesting.

Outline of the formal proof. Let A, B be independent fair bits, let A' be a separately named copy of A, and include a helper holding (A, B). The uses $(\varnothing, A)$ and $(\{A'\}, B)$ retain A through its copy, while the second context does not contain the first target's label. The formal observations embed these values in a common pair alphabet. For the XOR list, the second use would have to retain $\{A, B\}$ in context $\{A\}$; its missing entropy is $H(B \mid A)$. □

Thus a change of representation can preserve a cheap certificate, while the unrestricted XOR sharing example does not incorrectly receive a one-capacity retained-list bound.

Proposition 6.3 (Loss after a reveal)**.** Lean: `parityRank_isPolymatroid`, `parityRank_singleton`, `parityRank_of_pair_subset`, `helperLoss_strictly_smaller`, `helperLoss_increases_after_reveal` [`Sharpness.lean`]

Let A, B be independent fair bits and let $P = A \oplus B$. Use helper A twice, each time in empty context with target B, starting with $L_0 = \varnothing$. Before any common reveal,

$$D_A(\varnothing) = 0, \qquad \mathcal{L}(\varnothing) = \lambda.$$

After revealing P,

$$D_A(\{P\}) = \lambda.$$

Therefore helper-specific loss can be strictly smaller than whole-context loss, and it need not decrease under additional common conditioning.

Outline of the formal proof. Each of A, B, P has entropy λ and every pair has entropy 2λ; the Lean proof computes these entropies from the uniform law on two Boolean variables. Before revelation $I(A; B) = 0$, so the transition from $\{B\}$ to $\varnothing$ loses no information about A, although it loses one bit of context. After revelation $I(A; B \mid P) = \lambda$, so the same transition loses one bit of helper information. □

Before the reveal, helper A does not decode target B; the recovery residual must be retained. This example establishes a strict comparison of charges and a failed monotonicity claim, not a cheaper certificate for a correct transposer. In particular, $D_\varnothing$ cannot be used in place of D_Q after optimizing the choice of Q.

Proposition 6.4 (A revelation credit)**.** Lean: `oneBitRank_univ`, `oneBit_reveal_credit` [`Sharpness.lean`]

For a single fair selected bit X, take $b_X = \lambda$, no initial common context, and reveal X itself. Then

$$2J_Q - S_Q = \lambda, \qquad 2H(Y_Q) = 2\lambda.$$

Thus the effective revelation charge in (5.7) can be strictly smaller than the charge in (5.8).

Outline of the formal proof. The reveal supplies $J_Q = \lambda$ and leaves zero singleton entropy, giving $S_Q = \lambda$. The formal one-bit instance evaluates this difference exactly. □

The credit does not lower the unrestricted shared-helper coefficient: revealing a parity need not reduce either individual coordinate's uncertainty. The savings depend on the actual joint law.

7 Use in the transposition program

We keep the series' notation: d is the matrix dimension, $b(d) = \max\{1, \lceil \log_2 d \rceil\}$, and $L = d^2 b(d)$ is the input bit-length. The question (T) asks for a deterministic multitape transposer taking $O(L)$ time [2]. In the nats used here, the full input capacity is $L \log 2$.

What an impossibility application must establish. An application first chooses a distribution on the same input family throughout, selects coordinates, and proves a lower bound of the form (5.2). Lemma 5.7 permits a paid reveal and conditioning on the unselected input without assuming independence under that distribution. It must then provide a decomposable parent relation and actual helper values completing or partially completing the selected recoveries. The occurrence cover must preserve the local contexts (5.1).

For one chosen reveal Q, all remaining charges must be evaluated under that same distribution and common context. To contradict Theorem 5.3, it would suffice to prove

$$2\delta + 2J_Q + \Phi_Q + D_Q + E_Q < B + S_Q. \tag{7.1}$$

Here the strict upper bound must follow from the assumed machine class, not be supplied as an independent certificate assumption. A uniform lower bound on the positive gap in (7.1), as d grows, is a remaining target. No such gap for unrestricted native tape computations is established in this memo.

What a construction must account for. The same inequality is necessary for any compatible recovery scheme. Keeping earlier information in a new encoding is allowed. It can preserve semantic retention even when literal variable containment fails. If information is discarded, the helper-specific decrease is charged; if recovery is incomplete, E_Q is charged. Merely changing record names does not remove the occurrence or entropy obligations.

Records at different times are distinct observations, even if stored in the same tape cell. An equality or decoder between those observations may establish an entropy relation, but physical location alone does not. Nor may the contexts be enlarged by inserting earlier targets or whole tape blocks without proving that the revised recovery relation and occurrence cover meet their respective hypotheses. The memo's decoder lemmas certify information identities, not free computation.

Remaining questions. Three concrete tasks would establish further utility. First, identify a family of actual computations or recovery schemes for which the decomposition and occurrence-cover conditions can be obtained with quantitative bounds on all charges. Second, exhibit a substantial improvement over the applicable older bounds after their reveal and cover choices have also been optimized; the finite controls in Section 6 compare individual charges only. Third, find conditions under which the sharper D_Q can be bounded without solving the full joint information problem. A useful native instance of (7.1) would advance the impossibility program; compatible instances instead constrain where a construction can retain or replenish the required information. The general transposition question remains open.

A Index of Lean declarations

Local names are relative to `SoftRecovery`; imported names are fully qualified. The index is generated from the manuscript together with the axiom audit (Appendix B).

`Capacity.lean` `capacity_bound_of_residual`, `capacity_bound_with_credit`, `conditional_lower_from_deficit`, `mixed_record_bound`, `mixed_record_entropy_bound`, `optimized_record_bound`, `revealCredit_nonneg`, `threshold_bill`.

`ChainCapacity.lean` `RecordUseCover`, `RecordUseCover.entropy_bound`, `RecordUseCover.information_bound`, `RecordUseCover.residual_information_bound`, `RecordUseCover.retained_entropy_bound`, `allRecordUses`, `chain_record_bound_with_loss`, `coveredUses`, `optimized_chain_record_bound`, `optimized_chain_record_bound_with_loss`, `optimized_retained_chain_record_bound`, `recordUses`.

`ClassS.lean` `classSOn_soft`, `classS_soft`.

`Conditioning.lean` `entropy_conditionalRank_zero_of_recovery`.

`Decomposition.lean` `decomp_ineq_residual`, `decomp_ineq_soft`, `exact_card_sq`, `exact_decomp`, `exact_rateBound`, `piece_ineq_residual`, `totalDefect_le_residual`.

`Innovation.lean` `NestedUses.retained`, `RetainedUses`, `RetainedUses.contr`, `chainLoss_contr_le`, `helperLoss_le_chainLoss`, `increment_eq_helperInnovations_add`, `increment_eq_innovation_add`, `innovations_le_helperLoss`, `innovations_le_helper_capacity`, `innovations_le_with_loss`, `retained_innovations_le`.

`Observations.lean` `conditionalImage_isPolymatroid`, `conditional_soft`, `entropy_conditional_soft`, `image_isPolymatroid`.

`Sharpness.lean` `copied_contexts_not_nested`, `copied_contexts_retained`, `helperLoss_increases_after_reveal`, `helperLoss_strictly_smaller`, `oneBitRank_univ`, `oneBit_decomposable`, `oneBit_reveal_credit`, `parityRank_isPolymatroid`, `parityRank_of_pair_subset`, `parityRank_singleton`, `private_helper_coefficient_at_least_one`, `private_helper_recovers`, `residual_bound_attained`, `residual_coefficient_at_least_one`, `shared_helper_coefficient_at_least_two`, `shared_helper_one_fails`, `xor_uses_not_nested`, `xor_uses_not_retained`.

Imported declarations `HalfRate.Decomp`, `HalfRate.PieceCert`, `MotionDepthCodes.codeEntropy`.

B Verification and reuse

The new proofs are in the eight modules of `lean/SoftRecovery/`. The package imports the tenth paper through a relative dependency and, through it, the seventh and earlier papers. It reuses the original polymatroid, finite-entropy, nested-elimination, Class S, and XOR libraries. The original papers' proof sources are unchanged. In particular, the old exact decomposition theorem is re-derived as a corollary in this memo, rather than replaced in its original module.

Lean and Mathlib are pinned to v4.34.1. Every audited declaration may depend only on `propext`, `Classical.choice`, and `Quot.sound`. There are no proof placeholders, added axioms,

unsafe declarations, or kernel-bypassing evaluation proofs. No increased heartbeat limit is used. Finite Boolean checks use kernel-checked `decide`.

Clone the complete repository, preserving sibling directories. From the memo directory, run

```
python3 tools/lean_index.py
cd lean
lake exe cache get
lake build
lake env lean PaperAxioms.lean
```

The cache command is needed on a fresh checkout. The generator reads the manuscript's formal-status and inline declaration references to produce Appendix A. In the same operation it generates the audit file `lean/PaperAxioms.lean`, including every cited declaration, all public memo theorems, and the imported XOR witness. The existing `tools/generate_axioms.py` command is an alias for that generator. The Lean command checks the generated audit; the script itself proves no mathematical assertion.

In `paper/`, run `latexmk -pdf main.tex` to build this document. The manuscript is licensed under CC BY 4.0 and the Lean code under Apache 2.0. The bibliography pins the earlier papers to repository revision `10b94428cdc199369958661b3fae21096b40aca1`, whose imported sources supplied the extensions.

References

[1] Hans Brende and Claude Opus 5.5 (Anthropic). Curved motion, depth reduction, and storage codes for matrix transposition on tapes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 10b94428cdc199369958661b3fae21096b40aca1. URL: https://github.com/HansBrende/papers/tree/10b94428cdc199369958661b3fae21096b40aca1/Curved-Motion-Depth-Reduction-and-Storage-Codes-for-Matrix-Transposition-on-Tapes.

[2] Hans Brende and Claude Opus 5.5 (Anthropic). Matrix transposition and the tape coding conjecture. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 10b94428cdc199369958661b3fae21096b40aca1. URL: https://github.com/HansBrende/papers/tree/10b94428cdc199369958661b3fae21096b40aca1/Matrix-Transposition-and-the-Tape-Coding-Conjecture.

[3] Hans Brende and Claude Opus 5.5 (Anthropic). Nested elimination and half-rate bounds for storage codes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 10b94428cdc199369958661b3fae21096b40aca1. URL: https://github.com/HansBrende/papers/tree/10b94428cdc199369958661b3fae21096b40aca1/Nested-Elimination-and-Half-Rate-Bounds-for-Storage-Codes.

[4] Hans Brende and Gemini 4 Argon (Google DeepMind). Directed reciprocity and deep-nesting bounds for matrix transposition on tapes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 10b94428cdc199369958661b3fae21096b40aca1. URL: https://github.com/HansBrende/papers/tree/10b94428cdc199369958661b3fae21096b40aca1/Directed-Reciprocity-and-Deep-Nesting-Bounds-for-Matrix-Transposition-on-Tapes.

[5] Hans Brende and GPT-6 Astra (OpenAI). Local coding barriers and the limits of tape geometry for matrix transposition. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision 10b94428cdc199369958661b3fae21096b40aca1. URL: https://github.com/HansBrende/papers/tree/10b94428cdc199369958661b3fae21096b40aca1/Local-Coding-Barriers-and-the-Limits-of-Tape-Geometry-for-Matrix-Transposition.

[6] Ohad Elishco and Alexander Barg. Recoverable systems. arXiv:2010.00589v2, 2022. Version 2, 6 March 2022; first posted in 2020. URL: https://arxiv.org/abs/2010.00589v2.

[7] Satyajit Thakor, Alex Grant, and Terence Chan. Network coding capacity: A functional dependence bound. arXiv:0901.4612v1, 2009. URL: https://arxiv.org/abs/0901.4612v1.